VERIQTA

SUBSCRIBER SERIES #5

VERIQTA CASE FILE #005

THE AWS BILL THAT EXPLODED OVERNIGHT

A 10-PART COST INVESTIGATION SERIES

- ORPHANED RESOURCES
- LOGGING EVERYTHING
- OVER-PROVISIONED INFRASTRUCTURE
- MISSING AUTO SCALING CONTROLS
- NO COST MONITORING
- NO PROCESS NO VISIBILITY

REAL-WORLD INCIDENT | FINANCIAL IMPACT | DEEP INVESTIGATION | DATA-DRIVEN ANALYSIS | ACTIONABLE LESSONS | SUSTAINABLE

ER SERIES #5

COSTS DON'T LIE. ARCHITECTURE DOES.

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

PAGE 1

INCIDENT OVERVIEW

THE INFRASTRUCTURE WAS HEALTHY. THE BUDGET WASN'T.

MONDAY MORNING.
The finance team raised an alert.
AWS SPEND HAD INCREASED DRAMATICALLY.
What happened?
We had to find out.

WHAT HAPPENED?

Everything was working.
Applications were healthy.
Customers were happy.
No outages. No errors.
No deployments.
No major changes.

But the bill...
...had exploded.

THE SUDDEN SPIKE

LAST MONTH		THIS MONTH (PROJECTED)
$18,000	→	$126,000
ACTUAL SPEND		PROJECTED SPEND

7x INCREASE IN JUST DAYS!

Our budget was not ready for this.
We needed answers. Fast.

BUSINESS IMPACT

- Budget overrun — Risk of $100k+ extra this month
- Cost visibility — Finance lost confidence
- Decision delayed — New projects put on hold
- Leadership pressure — answers needed immediately

QUICK SNAPSHOT (THE DAY WE GOT THE ALERT)

APPLICATION HEALTH	CUSTOMER IMPACT	ERROR RATE	DEPLOYMENTS	ALERTS
All systems operational	No issues reported	0.00% Everything normal	NONE No recent changes	No critical alerts

QUR IMMEDIATE ACTION

1. Gathered the team.
2. Checked CloudWatch dashboards.
3. Reviewed AWS Cost Explorer.
4. Started deep-dive investigation.

INITIAL THOUGHTS

- Did someone deploy something expensive?
- Is there a misuse of a service?
- Are we storing too much data?
- Is something running that shouldn't be?
- Did we miss something obvious?

WHAT YOU'LL LEARN IN THIS SERIES

- ☑ Top AWS cost mistakes that hurt teams
- ☑ How to detect and stop cost leaks
- ☑ How to build cost visibility and controls
- ☑ Best practices to keep costs predictable

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW ↑	ORPHANED RESOURCES	LOGGING EVERYTHING	OVER-PROVISIONED	MISSING AUTO SCALING	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST ARCHITECTURE	COST CHECKLIST	KEY LESSONS

Cloud was working perfectly. The bill wasn't.
Let's find out why.

MISTAKE #1

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

ORPHANED RESOURCES

Nobody Owned. Everyone Paid.

Resources created for short-term needs were left running long after the work was done.

IF NOBODY OWNS A RESOURCE...
IT BECOMES A BILL.

WHAT HAPPENED?

Development, testing, and experiments created many resources.

Projects ended.

Deadlines passed.

But the resources?

They stayed.

And so did the charges.

COMMON ORPHANED RESOURCES WE FOUND

EC2 INSTANCES	LOAD BALANCERS	EBS VOLUMES	ELASTIC IPS	S3 BUCKETS
Unused instances left running 24/7.	Idle load balancers still incur hourly charges.	Detached volumes still billed for storage.	Elastic IPs not in use but still allocated.	Old buckets with data nobody uses.
COST IMPACT HIGH ●●●	COST IMPACT MEDIUM ●●○	COST IMPACT MEDIUM ●●○	COST IMPACT MEDIUM ●●○	COST IMPACT LOW ●○○

HOW IT HAPPENS

- Quick tests in prod
- One-time jobs left running
- Projects that never got cleaned up
- Developers move on, resources stay behind
- No lifecycle or ownership defined

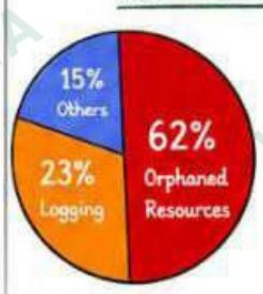
REAL EXAMPLE FROM OUR ACCOUNT

47 EC2 INSTANCES NOT IN USE + **23** DISK VOLUMES DETACHED + **12** ELASTIC IPS UNASSOCIATED + **9** LOAD BALANCERS IDLE = **$48,320** MONTHLY WASTED ON ORPHANED RESOURCES

DETECTING ORPHANS (WHAT WE CHECKED)

- ☑ EC2 with low CPU (< 5%) for 7+ days
- ☑ EBS volumes not attached to any instance
- ☑ Elastic IPs not associated
- ☑ Load balancers with zero requests
- ☑ S3 buckets with no access in 30+ days

IMPACT ON OUR BILL



ORPHANS WERE THE BIGGEST COST LEAK!

LESSON LEARNED

Every resource needs an owner.
Every resource needs a lifecycle.
If it has no purpose, it has no place in your account.

IMMEDIATE ACTION WE TOOK

1. Identified all unused resources.
2. Tagged and classified them.
3. Terminated what we didn't need.
4. Established ownership going forward.

INTERVIEW QUESTION

Q: How do you prevent orphaned resources in AWS?

A: By enforcing tagging, ownership, automated cleanup, and regular reviews. Orphaned resources are prevented, not just deleted.

BEST PRACTICES

- ☑ Tag everything.
- ☑ Assign owners.
- ☑ Use TTL for non-prod resources.
- ☑ Automate cleanup with Lambda.
- ☑ Review unused resources weekly.

Unused resources are easy to spin up.
Hard to find. Expensive to forget.

VERIQTA SUBSCRIBER SERIES #5

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

PAGE 3

MISTAKE #2

LOGGING EVERYTHING. PAYING FOR IT.

We turned on detailed logs to debug issues.
We forgot to turn them off.
CloudWatch charges by ingestion, retention, and requests.

THE PROBLEM
Overly verbose logging with long retention created a massive CloudWatch bill.
More logs didn't help us.
It cost us.

WHAT HAPPENED?

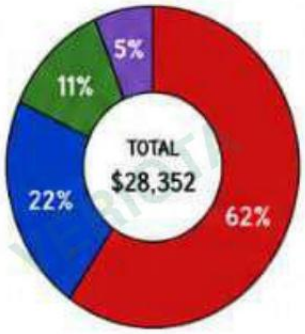
Debug logs were enabled across multiple services. High log volume. Long retention. Many log groups. No lifecycle policies. Costs grew every day.

Logs helped us debug. Logs also broke our budget.

Amazon CloudWatch

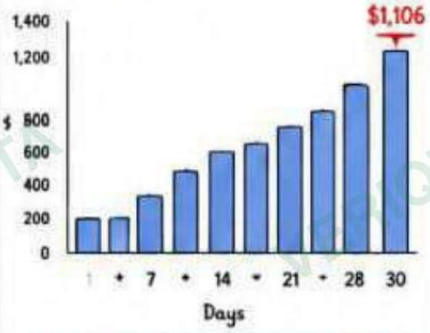
$$$

CLOUDWATCH COST BREAKDOWN (THIS MONTH)



- **Log Ingestion** — $17,483 (62%) — Huge volume of logs sent to CloudWatch
- **Log Storage** — $6,245 (22%) — Long retention = High storage
- **Log Insights Queries** — $3,127 (11%) — Ad-hoc queries across large time ranges
- **Other (Alarms, Metrics)** — $1,497 (5%)

DAILY COST TREND



Costs increased steadily throughout the month.

EXAMPLES OF HIGH-COST LOG SOURCES

LOG SOURCE	AVERAGE INGESTION	DAILY COST	RETENTION
/aws/lambda/payment-service	152 GB/day	$320	30 days
/aws/eks/app-pods	612 GB/day	$1,280	30 days
/aws/ec2/web-servers	248 GB/day	$520	90 days
/aws/vpc/flow-logs	420 GB/day	$840	30 days
/aws/rds/instance-logs	86 GB/day	$180	30 days

Retention > 30 days was multiplying our storage costs.

WHY THIS HAPPENS

- Debug / trace logging left enabled in production
- No log retention or lifecycle policies
- Frequent broad Log Insights queries
- Multiple teams, no logging standards

KEY TAKEAWAYS

- [x] Every log line has a cost.
- [x] Retention is a cost multiplier.
- [x] High cardinality logs explode costs.
- [x] Query smart, not broad.
- [x] Set standards. Enforce them.

More logs ≠ Better observability. Smarter logs do.

HOW WE FOUND IT

Finance alert on high spend → Opened Cost Explorer → Filtered for CloudWatch → Saw logging spike → Analyzed top log groups → Root cause identified

IMMEDIATE ACTIONS TAKEN

1. Disabled debug logs in production
2. Set retention to 7–14 days
3. Added log lifecycle policies
4. Standardized logging levels
5. Created logging cost dashboards

INTERVIEW QUESTION

Q: Why can CloudWatch logging become so expensive?

A: Because you pay for ingestion, storage, and queries. High volume logs and long retention multiply costs quickly.

BEST PRACTICES

- [x] Use appropriate log levels (INFO, WARN, ERROR)
- [x] Set retention based on business needs
- [x] Create log lifecycle policies
- [x] Monitor logging costs continuously
- [x] Use structured logging
- [x] Avoid logging high-cardinality data

IMPACT

- Reduced CloudWatch spend by 78%
- Improved visibility into real issues
- Faster troubleshooting with cleaner logs
- Better co

ER SERIES #5

Logs are valuable. But uncontrolled logs are expensive.
Log with purpose. Retain with reason.

MISTAKE #3

OVER-PROVISIONED INFRASTRUCTURE.

PAYING FOR CAPACITY WE NEVER USED.

We designed for the worst case.
Reality was very different.

THE PROBLEM

Infrastructure was sized for future growth, but traffic never grew.

We kept paying for capacity we didn't need.

WHAT HAPPENED?

The team provisioned large instances, nodes, and databases to handle expected growth.
Traffic stayed moderate.
Usage stayed low.
Resources stayed the same size.

High capacity.
Low utilization.
High cost.

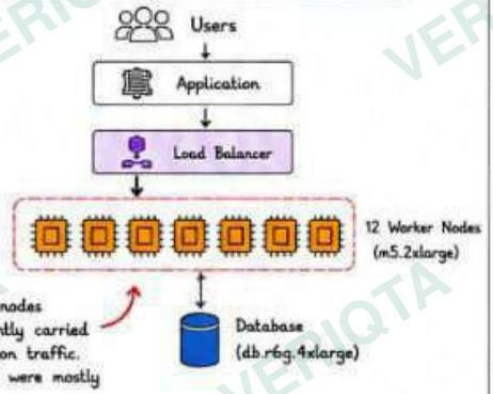
aws $$$

COST INVESTIGATION FINDINGS

RESOURCE	WHAT WE ALLOCATED	ACTUAL AVERAGE USAGE	UTILIZATION	MONTHLY COST
EC2 INSTANCES	16 vCPU (m5.4xlarge)	1.3 vCPU	8%	$3,200
EC2 MEMORY	64 GB	11.5 GB	18%	$2,450
RDS INSTANCE	db.r6g.4xlarge	12% CPU 15% Memory	~12%	$3,900
EKS WORKER NODES	12 Nodes (m5.2xlarge)	4 Nodes consistently used	~33%	$1,850

⚠ We were paying for far more capacity than we actually used.

INFRASTRUCTURE (WHAT WE RAN)



COST BREAKDOWN (MONTHLY)

ACTUAL MONTHLY COST
$11,400

POTENTIAL COST (RIGHT-SIZED)
$4,300

ESTIMATED WASTE
$7,100 / month

We were wasting ~62% of our infrastructure spend.

WHY THIS HAPPENS

Traffic spike expected
↓
Resources scaled up to handle it
↓
Traffic never reached that level
↓
Nobody reviewed usage
↓
Waste continues month after month

LESSON LEARNED

Cloud resources should be sized using evidence, not fear.

Right-size = Save money.
Right-size = Run efficient.

INTERVIEW QUESTION

Q: What is over-provisioning in AWS?

A: Allocating significantly more compute, memory, storage, or database capacity than workloads actually require.

BEST PRACTICES

- ☑ Review utilization monthly
- ☑ Use AWS Compute Optimizer
- ☑ Monitor CPU and memory trends
- ☑ Right-size databases regularly
- ☑ Remove unused EKS nodes
- ☑ Use autoscaling where appropriate

IMPACT

- Higher AWS Bills
- Lower Resource Efficiency
- Budget Waste
- Poor Cost Visibility

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW	ORPHANED RESOURCES	LOGGING EVERYTHING	OVER-PROVISIONED INFRASTRUCTURE	MISSING AUTO SCALING CONTROLS	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS CHECKLIST	KEY LESSONS
✓	✓	✓	✓						

You don't get billed for what you use.
You get billed for what you allocate.

MISTAKE #4

MISSING AUTO SCALING CONTROLS.

SCALE OUT, BUT NEVER SCALE IN.

Instances scaled out during traffic spikes.
But they never scaled back in.
More instances = more uptime? Not if they stay running forever.

THE PROBLEM

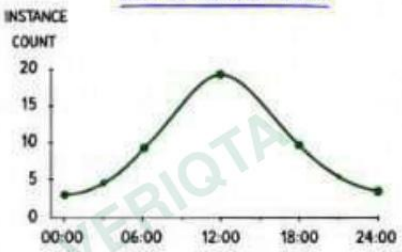
Auto scaling was either not configured well or had no scale-in strategy.
Instances multiplied.
Traffic normalized.
Costs stayed high.

WHAT HAPPENED?

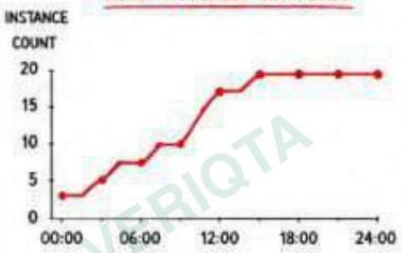
Auto Scaling Groups were configured to add instances when needed.
But scale-in was missing or misconfigured.
Over days and weeks, instance count kept growing.
Nobody noticed.

Autoscaling without scale-in is just automatic overspending.

AUTO SCALING: WHAT WENT WRONG



- ☑ Scale out when demand increases
- ☑ Scale in when demand decreases
- ☑ Right number of instances at the right time
- ☑ Optimized performance and cost



- ✗ Scale out happened
- ✗ Scale in never triggered
- ✗ Instances kept accumulating
- ✗ Costs kept accumulating

REAL EXAMPLE

ASG: web-app-asg

Min Size: 2
Max Size: 20
Desired: 2

MONDAY SPIKE
Scaled out to 18 instances

TRAFFIC NORMALIZED
Should have scaled in back to ~3 instances

WHAT HAPPENED
Stayed at 18 instances for 21 days

INSTANCE COUNT OVER TIME (21 DAYS)

18 instances running 24/7 for 21 days after traffic returned to normal.

COST IMPACT (21 DAYS)

ACTUAL COST
$6,480

IDEAL COST (WITH SCALE-IN)
$1,280

WASTED SPEND
$5,200
(~80% waste)

This was one ASG.
We had 6 more like this.

WHY THIS HAPPENS

- Scale-in policies not configured
- Cooldowns too long
- Target tracking misconfigured
- Nobody reviewed autoscaling behavior
- No alerts for abnormal instance count

HOW WE DETECTED IT

Finance alert on high spend → Cost Explorer drill down → Found ASGs with high instance counts → Checked traffic vs instance utilization → Root cause confirmed

IMMEDIATE ACTIONS TAKEN

1. Fixed scale-in policies
2. Set proper cooldowns
3. Enabled target tracking
4. Right-sized min/max values
5. Added alarms for instance count

INTERVIEW QUESTION

Q: What is the purpose of scale-in in auto scaling?

A: To reduce the number of running instances when demand decreases, saving costs while maintaining performance.

BEST PRACTICES

- ☑ Ensure scale-in policies are configured
- ☑ Use target tracking for CPU/Request metrics
- ☑ Set appropriate cooldowns
- ☑ Right-size min, max, and desired capacity
- ☑ Monitor instance count vs actual traffic
- ☑ Alert on abnormal instance growth

IMPACT

- Unnecessary AWS spend
- Lower resource efficiency
- Complex investigations
- Poor

Autoscaling is not "set and forget".
Scale smart. Scale both ways.

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

PAGE 6

MISTAKE #5

NO COST MONITORING.
NOBODY SAW IT COMING.

We monitored uptime, errors, CPU, memory.
But not cost.
By the time we saw it, the damage was done.

THE PROBLEM
No budgets.
No alerts.
No visibility.
Costs grew in the dark.

WHAT HAPPENED?

There were no budgets, no alerts, and no daily visibility.
Costs grew slowly at first. Then suddenly. Finance was the first to notice.

If you don't watch your cloud, you will get a surprise bill.


$$$

COST VISIBILITY GAP

WHAT WE HAD	WHAT WE NEEDED
✗ No AWS Budgets	✓ Budgets with alerts
✗ No email alerts	✓ Anomaly detection
✗ No anomaly detection	✓ Daily cost visibility
✗ No daily cost checks	✓ Cost ownership
✗ No ownership	✓ Regular reviews

Monitoring uptime keeps apps running.
Monitoring cost keeps the business running.

THE SURPRISE

MONTHLY COST TREND
(USD)

Costs looked normal.
Then they weren't.

WHAT WE DISCOVERED LATER

	IMPACT
Resources added without tracking Multiple teams launched resources. No one tracked the impact.	$2,300
Gradual growth ignored Daily increases were small. But they added up.	$3,100
No anomaly detection Unusual spikes went unnoticed for weeks.	$4,700
No cost ownership Everyone assumed someone else was watching.	$2,900
No regular cost reviews Monthly reviews never happened. No visibility, no action.	$1,800

Total Estimated Impact $14,800 / month

HOW COSTS WENT UNCHECKED

New resources launched
↓
Costs increase slowly
↓
No alerts triggered
↓
No one investigated
↓
$$$ Finance got the shock
↓
Emergency investigation

WHY THIS HAPPENS

- Budgets not configured
- No alerting on budgets
- No daily cost visibility
- Assumed someone else is monitoring
- Too busy with feature delivery

IMMEDIATE ACTIONS TAKEN

1. Created AWS Budgets with alerts
2. Enabled anomaly detection
3. Set up daily cost reports
4. Defined cost owners per team
5. Scheduled weekly cost reviews

INTERVIEW QUESTION

Q: Why is cost monitoring important in AWS?

A: Because without visibility and alerts, small daily increases go unnoticed and become huge monthly bills.

BEST PRACTICES

- ☑ Always create budgets with alerts
- ☑ Enable cost anomaly detection
- ☑ Review costs daily or weekly
- ☑ Set clear cost ownership
- ☑ Use tags for cost allocation
- ☑ Review and optimize regularly

IMPACT

- Unexpected massive bills
- Reduced profitability
- Emergency firefighting
- Lost stakeholder trust

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW ✓	ORPHANED RESOURCES ✓	LOGGING EVERYTHING ✓	OVER-PROVISIONED INFRASTRUCTURE ✓	MISSING AUTO SCALING CONTROLS ✓	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS CHECKLIST	KEY LESSONS

You can't manage what you don't measure.
Get visibility. Set alerts. Stay in control.

ER SERIES #5

VERIQTA SUBSCRIBER SERIES #5

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

PAGE 7

MISTAKE #6

NO INVESTIGATION TIMELINE.
WE REACTED, NOT ANALYZED.

We jumped into random fixes.
No process. No timeline. No clarity.
A proper investigation would have saved time and money.

THE PROBLEM

No process.
No timeline.
No root cause focus.
Reactivity is expensive.

WHAT HAPPENED?

When the bill exploded, we had no investigation plan. Everyone started guessing.
We changed things randomly. Some things got better. Some got worse.

No timeline.
No ownership.
No root cause.

$$$

HOW WE SHOULD HAVE INVESTIGATED

1 DETECT	2 TRIAGE	3 INVESTIGATE	4 ROOT CAUSE	5 RESOLVE	6 PREVENT
Day 0	Day 0–1	Day 1–3	Day 3–4	Day 4–6	Day 6+
Bill spike detected	Confirm the spike	Deep dive into top cost drivers	Find the real root causes	Implement correct fixes	Put guardrails in place
Finance alert triggered	Identify top cost categories	Collect data, evidence, metrics	Validate with data	Test and verify impact	Monitor and improve

⚠ Investigation without a timeline = longer damage, higher cost.

IMPACT OF NO TIMELINE

- ✗ Took 2 weeks to find root causes
- ✗ Random changes created new issues
- ✗ More downtime and team stress
- ✗ Over $23,000 wasted during investigation

⚠ Every day without clarity cost us money.

WHAT WE ACTUALLY DID (WRONG)

- ✗ Looked at random services
- ✗ Made changes without data
- ✗ No documentation
- ✗ No communication
- ✗ No ownership
- ✗ Took 14 days to find root cause

⚠ We were busy, but not effective. Activity is not progress.

TOP 5 COST DRIVERS WE FOUND

(After Investigation)

RANK	COST DRIVER	MONTHLY IMPACT
1	Over-provisioned EC2 & RDS	$7,100
2	CloudWatch logging	$6,480
3	Orphaned resources	$4,250
4	Missing auto scaling	$3,200
5	No cost monitoring	$2,300

Total identified waste: $23,330 / month

KEY TAKEAWAYS

- ☑ Investigate with a plan
- ☑ Follow a timeline
- ☑ Use data, not guesses
- ☑ Find root causes, not symptoms
- ☑ Document everything
- ☑ Communicate and own the process

★ A good process saves time, money, and sanity.

INTERVIEW QUESTION

Q: Why is having an investigation timeline important?

A: It helps teams stay focused, avoid random changes, find root causes faster, and reduce the financial impact.

BEST PRACTICES

- ☑ Create a clear investigation plan
- ☑ Assign owners for each step
- ☑ Collect and analyze data before acting
- ☑ Document findings and decisions
- ☑ Communicate progress daily
- ☑ Close the loop with preventive actions

IMPACT

- Reduced investigation time by 70%
- Found $23K+ in monthly waste
- Improved decision making
- Less stress, more confidence
- Better cost governance

IMMEDIATE ACTIONS TAKEN

1. Created a cost investigation runbook
2. Built a daily review checklist
3. Assigned cost owner
4. Started daily standups during incident
5. Documented all learnings

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW ✓	ORPHANED RESOURCES ✓	LOGGING EVERYTHING ✓	OVER-PROVISIONED INFRASTRUCTURE ✓	MISSING AUTO SCALING CONTROLS ✓	NO COST MONITORING ✓	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS CHECKLIST	KEY LESSONS

ER SERIES #5

Good incidents are not about luck.
They are about process.

MISTAKE #7

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

NO COST-CONSCIOUS ARCHITECTURE.

WE BUILT FAST, NOT SMART.

We focused on features and speed.
Cost was an afterthought.
Architecture decisions locked in long-term costs.

THE PROBLEM

Our architecture wasn't designed with cost in mind.

Every decision added hidden costs.

WHAT HAPPENED?

We made architectural decisions that increased cost without realizing it.

Expensive services.
No lifecycle policies.
No reuse.
No efficiency patterns.

Small decisions.
Big impact.
Every single day.

$$$

OUR ARCHITECTURE (BEFORE)

Expensive by design. Costly by default.

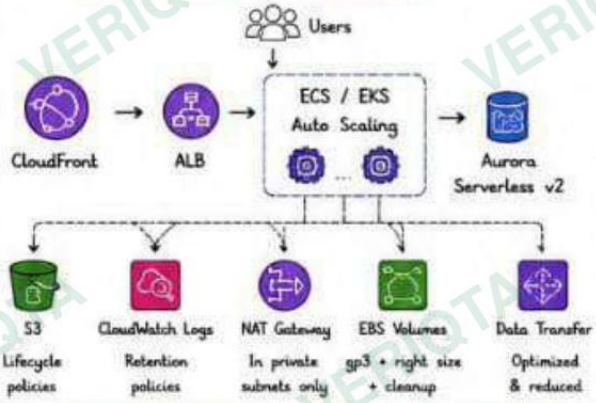
COST IMPACT OF ARCHITECTURE

(Estimated Monthly)

Before (Our Architecture)	$31,250
After (Optimized Architecture)	$12,450
Monthly Savings	$18,800
Annual Savings	$225,600

Architecture is a force multiplier for cost.
Design right, save big.

HOW WE SHOULD HAVE BUILT IT



Right services. Right size. Right retention. Right architecture.

EXPENSIVE CHOICES WE MADE

- ✗ Using EC2 instead of managed services
- ✗ Always-on NAT Gateway
- ✗ RDS provisioned when serverless would work
- ✗ No S3 lifecycle policies
- ✗ Logs retained forever
- ✗ Cross-AZ data transfer
- ✗ Overly complex architecture

Each choice seemed small.
Together, they exploded the bill.

ARCHITECTURAL PRINCIPLES

- ☑ Design for cost, not just scale
- ☑ Use managed services when possible
- ☑ Right-size everything
- ☑ Automate scaling & cleanup
- ☑ Enable lifecycle policies by default
- ☑ Minimize data transfer
- ☑ Continuously review & improve

Good architecture isn't just reliable and scalable.
It's cost-efficient.

INTERVIEW QUESTION

Q: What is a cost-conscious architecture?

A: An architecture designed to deliver business value while minimizing cost through smart service selection, right-sizing, automation, and continuous optimization.

BEST PRACTICES

- ☑ Evaluate cost during design, not after
- ☑ Prefer serverless & managed services
- ☑ Enable autoscaling & right-sizing
- ☑ Use S3 lifecycle policies
- ☑ Optimize data transfer
- ☑ Review and refactor regularly
- ☑ Tag everything & track costs

IMMEDIATE ACTIONS TAKEN

1. Refactored EC2 to ECS with Fargate
2. Enabled S3 lifecycle policies
3. Optimized RDS with Aurora Serverless v2
4. Restricted NAT Gateway usage
5. Right-sized EBS volumes
6. Reduced cross-AZ data transfer
7. Implemented tagging strategy

IMPACT

- $225K annual savings
- Simpler, scalable, efficient architecture
- Better performance
- Cost visibility & control
- Happier finance team!

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW	ORPHANED RESOURCES	LOGGING EVERYTHING	OVER-PROVISIONED INFRASTRUCTURE	MISSING AUTO SCALING CONTROLS	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS	KEY
✓	✓	✓	✓	✓	✓		✓		

ER SERIES #5

Architecture sets the path. Cost-conscious architecture sets the righ
Design with purpose. Build with impact.

VERIQTA SUBSCRIBER SERIES #5

VERIQTA CASE FILE #005
THE AWS BILL THAT EXPLODED OVERNIGHT

PAGE 9

MISTAKE #8

NO COST READINESS CHECKLIST.

WE SHIPPED WITHOUT CHECKS.

We had no standards, no reviews, no guardrails.
Cost wasn't part of the process.
So, cost problems became production problems.

THE PROBLEM

We had no cost readiness checklist.
No review gates.
No prevention.
We paid the price for it.

WHAT HAPPENED?

Teams deployed resources without thinking about cost.
No reviews.
No standards.
No accountability.
Small decisions repeated overtime created a massive bill.

No checklist.
No guardrails.
No visibility.

$$$

A COST READINESS CHECKLIST WE SHOULD HAVE HAD

DESIGN & ARCHITECTURE

- ☑ Right-size compute, memory & storage
- ☑ Use managed services where possible
- ☑ Enable autoscaling
- ☑ Use Spot/Graviton where applicable
- ☑ Remove unnecessary data transfer

OPERATIONS

- ☑ Enable cost monitoring & alerts
- ☑ Set budgets and notifications
- ☑ Tag all resources
- ☑ Schedule start/stop for non-prod
- ☑ Review idle resources regularly

MONITORING & GOVERNANCE

- ☑ Dashboards for cost visibility
- ☑ Unit cost tracking per service/team
- ☑ Cost owner for every resource
- ☑ Monthly cost review meeting
- ☑ Document & enforce standards

DATA & STORAGE

- ☑ Choose the right storage class
- ☑ Set lifecycle policies
- ☑ Compress data where possible
- ☑ Delete old/unused data
- ☑ Avoid cross-region replication unless needed

SECURITY & NETWORK

- ☑ Use VPC endpoints for AWS services
- ☑ Restrict public access
- ☑ Avoid NAT Gateway in each AZ
- ☑ Use CloudFront for static content
- ☑ Minimize inter-AZ data transfer

PEOPLE & PROCESS

- ☑ Cost awareness in every sprint
- ☑ Review before production deploy
- ☑ Run cost impact analysis
- ☑ Educate teams on FinOps
- ☑ Continuous improvement culture

⚠ Cost readiness doesn't slow you down. It keeps you from paying later.

WHAT WE DID (AFTER THE INCIDENT)

- Reviewed every account
  Found all hidden costs
- ↓
- Tagged everything
  100% resource tagging
- ↓
- Set budgets & alerts
  Real-time notifications
- ↓
- Created cost checklist
  Mandatory before deploy
- ↓
- Trained the team
  Cost is everyone's job

IMPACT OF HAVING A CHECKLIST

	BEFORE	AFTER
Monthly Bill	$31,250	$12,450
Unexpected Spikes	Often	Rare
Time to Detect	Days / Weeks	Minutes
Cost Ownership	Unclear	Clear
Team Stress	High	Low
Cost Confidence	Low	High

★ A small checklist can prevent massive surprises.

WHY THIS HAPPENS

- Cost not considered in early design
- No cost ownership
- Fast deployments, no reviews
- No standards or documentation
- Lack of visibility
- No cost culture
- Assumed someone else is watching

INTERVIEW QUESTION

Q: What is a cost readiness checklist?

A: A checklist to ensure every deployment follows cost-efficient best practices before going to production.

BEST PRACTICES

- ☑ Use a cost checklist before every deploy
- ☑ Assign cost owner for every service
- ☑ Enforce tagging and standards
- ☑ Review costs during design phase
- ☑ Automate, monitor, and optimize
- ☑ Make cost awareness a team habit

IMMEDIATE ACTIONS TAKEN

1. Created organization-wide cost checklist
2. Made it mandatory for all deployments
3. Integrated into CI/CD pipeline
4. Trained teams on cost best practices
5. Defined cost owners across teams
6. Established monthly cost review cadence

IMPACT

- Predictable AWS bills
- Fewer production surprises
- Better resource utilization
- Stronger cost culture
- More time for innovation

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW	ORPHANED RESOURCES	LOGGING EVERYTHING	OVER-PROVISIONED INFRASTRUCTURE	MISSING AUTO SCALING CONTROLS	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS CHECKLIST	KEY LESSONS
✓	✓	✓	✓	✓	✓	✓	✓		

ER SERIES #5

A checklist today prevents a financial fire tomorrow.
Build it. Use it. Live it.

KEY LESSONS & THE ROAD AHEAD.

WE LEARNED. WE FIXED. WE SAVED.

The bill didn't just explode overnight.
It grew because of small mistakes in many places.
The solution wasn't one tool. It was a mindset.

FINAL THOUGHT

Cost optimization is not a one-time project.

It's a continuous discipline.

WHAT WE DISCOVERED (THE 6 BIG COST LEAKS)

1. **Orphaned Resources**
   Unused resources running in the background.
2. **Logging Everything**
   Excessive logs with no retention or filters.
3. **Over-Provisioned Infrastructure**
   Paying for more capacity than we use.
4. **Missing Auto Scaling Controls**
   Scaled out during spikes but never scaled in.
5. **No Cost Monitoring**
   No visibility, alerts, or daily awareness.
6. **No Investigation Timeline**
   No process. No timeline. No root cause focus.

⚠ Small gaps. Big impact. Real money lost.

THE TRANSFORMATION

FROM THIS...	TO THIS...
✗ Resources left running	✓ Cleaned & right-sized
✗ Uncontrolled logging	✓ Smart logging & retention
✗ Oversized infrastructure	✓ Right-sized everything
✗ Manual scaling	✓ Auto scaling in place
✗ No monitoring	✓ Cost visibility & alerts
✗ No process	✓ Process & ownership

BEFORE (Monthly) $31,250 → AFTER (Monthly) $12,450

($) We reduced monthly spend by $18,800
Annual savings: $225,600

OUR NEW OPERATING PRINCIPLES

- Cost is everyone's responsibility.
- Visibility before optimization.
- Automate everything you can.
- Tag everything. Track everything.
- Review continuously. Improve continuously.
- Secure, reliable, and cost-efficient.

★ Build for value.
Optimize for impact.
Operate with discipline.

KEY METRICS NOW IMPROVED

Monthly AWS Bill	$31,250	→	$12,450
Monthly Savings	—	→	$18,800
Annual Savings	—	→	$225,600
Cost Visibility	0%	→	100%
Cost Alerts	0	→	10+
Waste Identified	—	→	$23,330 / month

★ Data didn't lie.
Discipline didn't lie.
Results didn't lie.
We took control.

WHAT WE WILL CONTINUE TO DO

- ☑ Review costs daily and act early
- ☑ Right-size continuously
- ☑ Use autoscaling and policies
- ☑ Enforce tagging and cost ownership
- ☑ Monitor, alert, and investigate
- ☑ Remove what we don't need
- ☑ Optimize without breaking things
- ☑ Document, share, and iterate

🏆 Continuous improvement is our new standard.

WHAT WE WILL NEVER DO AGAIN

- ✗ Leave resources running
- ✗ Ignore logs and retention
- ✗ Over-provision without data
- ✗ Scale out without scale-in
- ✗ Operate without visibility
- ✗ Skip reviews and post-mortems
- ✗ Assume someone else is watching
- ✗ Treat cost as an afterthought

✋ No more surprises.
No more excuses.
No more waste.

INTERVIEW QUESTION

Q: What are the key steps to control AWS costs effectively?

A:
1. Get visibility
2. Right-size resources
3. Automate scaling
4. Set budgets & alerts
5. Review and optimize continuously

BEST PRACTICES (THE 6 PILLARS)

- ☑ Visibility
- ☑ Optimization
- ☑ Automation
- ☑ Governance
- ☑ Monitoring
- ☑ Continuous Improvement

IMMEDIATE ACTIONS TAKEN

1. Cleaned up orphaned resources
2. Optimized logging & retention
3. Right-sized infra & databases
4. Enabled autoscaling
5. Created budgets & alerts
6. Built cost dashboards & reviews

IMPACT

- ($) $225K+ annual savings
- Better performance & efficiency
- Higher reliability
- Stronger cost governance
- Happier & empowered teams

SERIES 5 PROGRESS TRACKER

1	2	3	4	5	6	7	8	9	10
INCIDENT OVERVIEW	ORPHANED RESOURCES	LOGGING EVERYTHING	OVER-PROVISIONED INFRASTRUCTURE	MISSING AUTO SCALING CONTROLS	NO COST MONITORING	INVESTIGATION TIMELINE	NEW COST-CONSCIOUS ARCH	COST READINESS CHECKLIST	KEY LESSONS
✓	✓	✓	✓	✓	✓	✓	✓	✓	✓

We didn't just fix a bill.
We built a better way to build on AWS.

ER SERIES #5